



Università di Genova

High Performance Computing Report CUDA

Andrea Cattarinich 5137057

December 4, 2025

Contents

1	Architectures	2
1.1	HP Envy 17-ae103nl (Laptop)	2
1.2	Google Colab	2
2	Hotspot Analysis	3
2.1	Code Description	4
2.2	Project Structure	7
2.3	Profiling	7
3	CUDA Parallelization	9
3.1	Sequential section	9
3.2	Code	9
3.3	Compilation	10
3.4	Execution	11
3.5	Profiling	11
3.6	Results	13
4	Conclusions	13
4.1	Realistic System Simulation	13
4.2	Considerations	14

1 Architectures

For this project, I used two workstations: my personal laptop and the Google Colab environment

1.1 HP Envy 17-ae103nl (Laptop)

CPU

- **Model:** Intel Core i7-6500U @ 2.50 GHz
- **Architecture:** x86_64
- **Physical cores:** 2
- **Logical threads:** 4 (Hyper-Threading)

GPU

- **Model:** NVIDIA GeForce GTX 950M
- **CUDA Compute Capability:** 5.0
- **Driver version:** 572.16
- **CUDA Version:** 12.8 (*CUDA Toolkit 12.x*)

Software and Setup:

- Visual Studio Community 2022 - Workload: Desktop development with C++
- CUDA Toolkit installed manually to enable GPU programming
- WSL2
- NVIDIA drivers (version 572.16) installed for WSL2 from NVIDIA official website

1.2 Google Colab

CPU

- **Model:** Intel(R) Xeon(R) CPU @ 2.20GHz
- **Architecture:** x86_64
- **Physical cores:** 1
- **Logical threads:** 2 (Hyper-Threading)
- **Socket(s):** 1

GPU

- **Model:** NVIDIA Tesla T4
- **CUDA Compute Capability:** 7.5
- **Driver version:** 550.54.15
- **CUDA Version:** 12.8 (*CUDA Toolkit 12.x*)

2 Hotspot Analysis

Heat conduction (or heat diffusion) is the process by which thermal energy spreads within a material due to temperature differences. In two dimensions, this phenomenon can be represented as heat spreading across a plate or surface over time. Numerical simulations of heat conduction are widely used in engineering, physics, and high-performance computing to predict temperature distribution and analyze thermal effects.

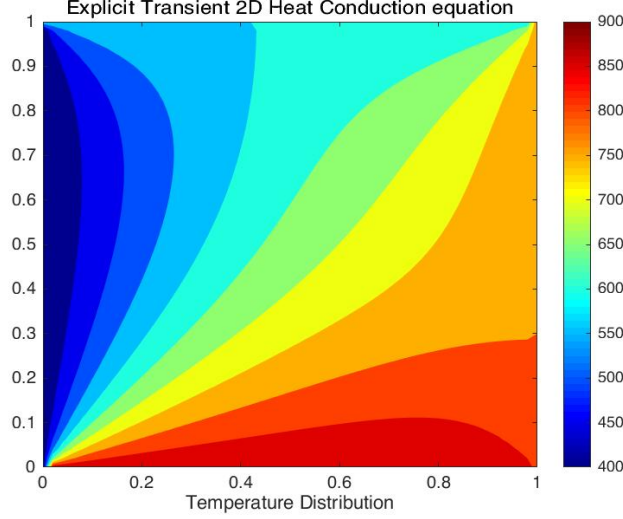


Figure 1: Illustration of 2D heat conduction simulation.

The formula to represent 2D heat conduction is given as follows:

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) \quad (1)$$

where:

- T is temperature;
- t is time;
- α is the thermal diffusivity;
- (x, y) are points in a grid.

To solve this problem numerically, one possible finite difference approximation is:

$$\frac{\Delta T}{\Delta t} = \frac{T_{i,j}^{n+1} - T_{i,j}^n}{\Delta t} = \alpha \left(\frac{T_{i+1,j}^n - 2T_{i,j}^n + T_{i-1,j}^n}{(\Delta x)^2} + \frac{T_{i,j+1}^n - 2T_{i,j}^n + T_{i,j-1}^n}{(\Delta y)^2} \right) \quad (2)$$

where:

- Δt is the timestep;
- i, j are indices in the grid;
- n indicates the current timestep, so $T_{i,j}^n$ is the temperature at point (i, j) at time $t = n\Delta t$, and $T_{i,j}^{n+1}$ is the temperature at the next timestep $t = (n+1)\Delta t$.

2.1 Code Description

This program simulates the diffusion of heat on a two-dimensional plate using the finite difference method. It includes two main implementations:

1. **Reference CPU version** - `step_kernel_ref()` - serial implementation
2. **Modified version** - `step_kernel_mod()` - intended for CUDA acceleration

Both implementations are designed to produce the same results.

Data Layout

The simulation uses a grid of size `ni × nj` (columns × rows).

Temperature values are stored in a **1D array**. A macro is used to convert 2D indices to linear index:

```
1 #define I2D(num, c, r) ((r)*(num)+(c))
```

Initialization

In the beginning, the boundary points of the grid are assigned with initial *random* temperatures, while the inner points update their temperature iteratively according to the formula above.

```
1 for (int i = 0; i < ni * nj; ++i) {  
2     t1_ref[i] = t2_ref[i] =  
3     t1[i]     = t2[i]     =  
4         (float)rand() / (float)(RAND_MAX / 100.0f);  
5 }
```

Realistic System Inizialization

In addition to the synthetic configurations used, I also decided to simulate a more realistic scenario in order to visually inspect the physical behavior of the system. Specifically, I initialized the domain by placing a high-temperature square region at the center of the grid, representing a localized heat source. All remaining points were initialized with a low random temperature, mimicking environmental thermal noise. The initialization code is reported below:

```
1 // Hot square parameters  
2 int hot_size = 40;      // side length of the hot square  
3 int hot_temp = 80;      // temperature of the hot region  
4 int hot_i0 = ni/2 - hot_size/2;    // center coordinate in i  
5 int hot_j0 = nj/2 - hot_size/2;    // center coordinate in j  
6  
7 for(int j = 0; j < nj; j++){  
8     for(int i = 0; i < ni; i++){  
9         int idx = I2D(ni, i, j);  
10  
11         if(i >= hot_i0 && i < hot_i0 + hot_size &&
```

```

12         j >= hot_j0 && j < hot_j0 + hot_size)
13     {
14         // heat source
15         temp1[idx] = temp2[idx] = (float)hot_temp;
16     }
17     else
18     {
19         // random background
20         temp1[idx] = temp2[idx] = (float)rand()/RAND_MAX*30.0f;
21     }
22 }
23 }

```

The source code can be found in `src/image.cu` and the script used for the visualization was done with Matlab (more details in: `src/realistic_simulation.m`)

The visual outcome of this configuration is discussed in Subsection 4.1.

Execution

The core of the simulation is inside the `step_kernel()` function, which implements the approximation described in formula (2), shown below.

```
1 void step_kernel_mod(int ni, int nj, float fact, float* temp_in,
2   float* temp_out)
3 {
4   int i00, im10, ip10, i0m1, i0p1;
5   float d2tdx2, d2tdy2;
6
7   // loop over all points in domain (except boundary)
8   for ( int j=1; j < nj-1; j++ ) {
9     for ( int i=1; i < ni-1; i++ ) {
10      // find indices into linear memory
11      // for central point and neighbours
12      i00 = I2D(ni, i, j);
13      im10 = I2D(ni, i-1, j);
14      ip10 = I2D(ni, i+1, j);
15      i0m1 = I2D(ni, i, j-1);
16      i0p1 = I2D(ni, i, j+1);
17
18      // evaluate derivatives
19      d2tdx2 = temp_in[im10]-2*temp_in[i00]+temp_in[ip10];
20      d2tdy2 = temp_in[i0m1]-2*temp_in[i00]+temp_in[i0p1];
21
22      // update temperatures
23      temp_out[i00] = temp_in[i00]+fact*(d2tdx2 + d2tdy2);
24    }
25  }
```

Note: The formula corresponds (2) to lines 18–23 in the above code.

Verification

After all timesteps, `temp1_ref` holds the CPU reference results, while `temp1` holds the modified version results. If the error exceed a threshold (e.g. 0.0005), the simulation is considered correct.

```
1 float maxError = 0;
2
3 for( int i = 0; i < ni*nj; ++i ) {
4   if (abs(temp1[i]-temp1_ref[i]) > maxError) { maxError = abs(
5     temp1[i]-temp1_ref[i]); }
6 }
7
8 // Check and see if our maxError is greater than an error bound
9 if (maxError > 0.0005f)
10   printf("The error is NOT within acceptable bounds.");
11 else
12   printf("The error is within acceptable bounds");
```

2.2 Project Structure

The `homework2` project is organized into four main directories:

- **build**: contains the compiled executables.
- **reports**: stores the reports generated with the `-qopt-report` flag.
- **src**: includes the source code files.
- **metrics**: scripts that run the executables in **build** and save performance metrics.

2.3 Profiling

For performance profiling, I chose to use the university workstation, using Intel’s profiling tools: **VTune** and **Advisor**. The code was compiled using the Intel Compiler (**ICX**) with the following flags: `-O3 -g -xHost` to enable high optimization, generate debug information, and target the host CPU architecture.

Simulation Parameters

The analysis was conducted considering data size resulting in a sequential execution time of ~ 30 seconds, by setting:

- $nstep = 200$
- $ni = 20'000$
- $nj = 20'000$
- $size = ni \times nj = 400'000'000$

Compiler Reports

In addition to runtime profiling, I also examined the compiler optimization reports generated with the flag `-qopt-report`. These reports provide insights on which loops and functions the compiler was able to optimize or vectorize.

Advisor Roofline Analysis

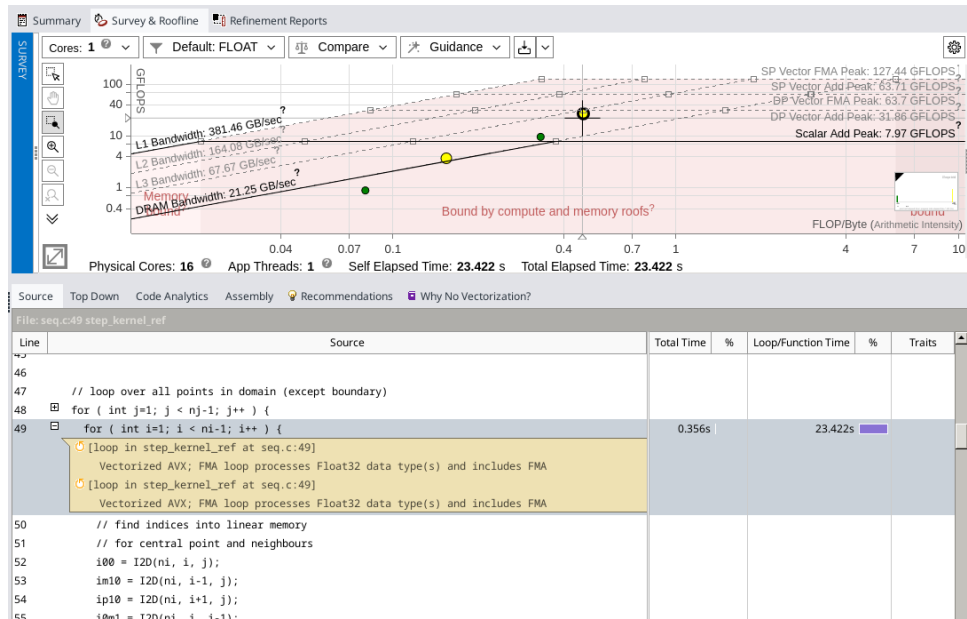


Figure 2: Roofline analysis of the sequential baseline code.

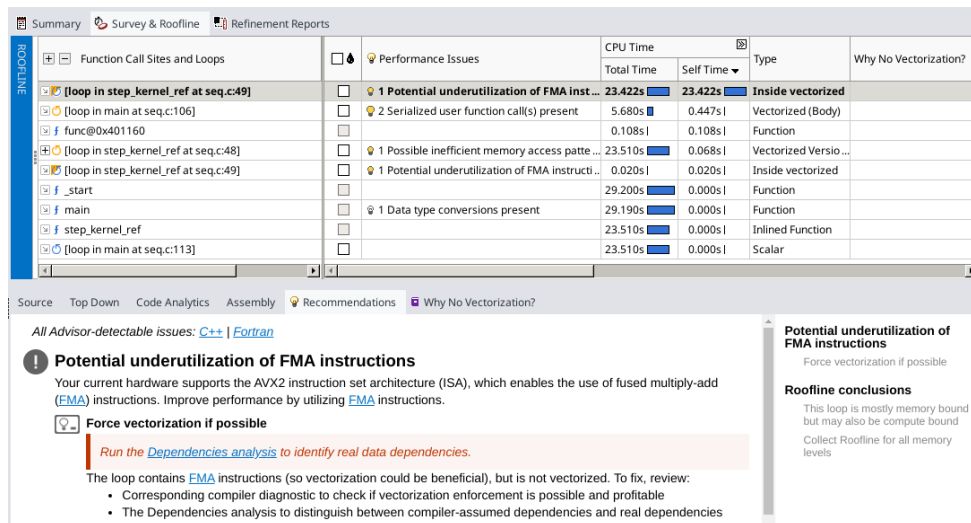


Figure 3: Advisor Survey.

3 CUDA Parallelization

3.1 Sequential section

I initially decided to run the tests on my personal PC rather than on the university workstation or on Google Colab, in order to avoid limitations related to temporary sessions and restricted computational resources. This allowed me to maintain full control over the compilation process and the execution of the tests during the initial experiments with CUDA.

The simulation parameters used are:

- $nstep = 200$
- $ni = 15000$
- $nj = 15000$
- $size = ni \times nj = 900\,000\,000$

I compiled the sequential code located in `src/cuda.c` using the following options:

```
gcc -O3 src/cuda.c -o build/cuda
```

The best execution time obtained (37 s) is used as the sequential baseline for computing the speedup of the GPU implementation.

3.2 Code

The parallelization was implemented using CUDA to execute the time-step computations on the GPU. The main kernel is `step_kernel_mod`, which replicates the behavior of the sequential function `step_kernel_ref`. You can find the code in `src/cuda.cu`.

Kernel structure:

- Each thread processes one point of the 2D grid.
- The global indices i and j are computed from `threadIdx` and `blockIdx`.

Memory management:

- Allocation of host and device memory.
- Transfer of the initial data from host to device.
- Transfer of the results from device to host at the end of the simulation.

Error checking:

- After each kernel launch, `cudaGetLastError()` is invoked.
- The maximum error relative to the CPU reference is computed to check correctness.

```
1 __global__ void step_kernel_mod(int ni, int nj, float fact, float
2   * temp_in, float* temp_out)
3 {
4   int i00, im10, ip10, i0m1, i0p1;
5   float d2tdx2, d2tdy2;
6
7   int i = threadIdx.x + blockIdx.x * blockDim.x;
8   int j = threadIdx.y + blockIdx.y * blockDim.y;
9
10  if(i > 0 && i < ni-1 && j > 0 && j < nj-1){
11    // find indices into linear memory
12    // for central point and neighbours
13    i00 = I2D(ni, i, j);
14    im10 = I2D(ni, i-1, j);
15    ip10 = I2D(ni, i+1, j);
16    i0m1 = I2D(ni, i, j-1);
17    i0p1 = I2D(ni, i, j+1);
18
19    // evaluate derivatives
20    d2tdx2 = temp_in[im10]-2*temp_in[i00]+temp_in[ip10];
21    d2tdy2 = temp_in[i0m1]-2*temp_in[i00]+temp_in[i0p1];
22
23    // update temperatures
24    temp_out[i00] = temp_in[i00]+fact*(d2tdx2 + d2tdy2);
25  }
```

3.3 Compilation

CUDA compilation was executed with the command:

```
nvcc -O3 -arch=sm_50 cuda.cu -o cuda.exe -DDEBUG=0
```

Significato dei flag:

- `-O3`: maximum optimization.
- `-arch=sm_50`: target GPU architecture.
- `-DDEBUG=0`: disables the `DEBUG` directives.

3.4 Execution

For the CUDA kernel, the chosen values for `threadsPerBlock` and `numBlocks` were the following.

```
1      dim3 threadsPerBlock(16, 16);
2      dim3 numBlocks((ni + 15) / 16, (nj + 15) / 16);
3
4      // ---- CUDA events start ----
5      cudaEvent_t start, stop;
6      cudaEventCreate(&start);
7      cudaEventCreate(&stop);
8
9      // Execute the modified version using same data
10     printf("Execute GPU kernel...\n");
11     cudaEventRecord(start, 0); // start timing
12
13     for (istep = 0; istep < nstep; istep++) {
14         step_kernel_mod <<<numBlocks, threadsPerBlock>>> (ni, nj,
15             tfac, d_temp1, d_temp2);
16         cudaDeviceSynchronize();
17
18         cudaError_t err = cudaGetLastError();
19         if (err != cudaSuccess)
20             printf("CUDA Error: %s\n", cudaGetErrorString(err));
21
22         float* d_tmp = d_temp1;
23         d_temp1 = d_temp2;
24         d_temp2 = d_tmp;
25     }
26
27     cudaEventRecord(stop, 0); // stop timing
28     cudaEventSynchronize(stop);
```

Each 16×16 thread block computes a portion of the grid, while the blocks are arranged to cover the entire matrix.

Timing was performed using `cudaEvent_t` to measure the execution time of the GPU kernel.

3.5 Profiling

Command: `nvprof build/cuda`

950M:

Execute the CPU-only reference version...

- Elapsed time: 0.188 s

Execute GPU kernel...

- Elapsed time: 140.515 ms

Max Error = 0.000008

The Max Error of 0.00001 is within ACCEPTABLE bounds.

- Total elapsed time: 2.460 s

==768== Profiling application: build/cuda

==768== Profiling result:

Type	Time(%)	Time	Calls	Avg	Min	Max	Name
GPU:	72.95%	202.16ms	2	101.08ms	2.7665ms	199.40ms	[CUDA memcpy HtoD]

	24.93%	69.101ms	200	345.50us	342.59us	355.94us	step_kernel_mod(int, int, float, float*, float*)
	2.12%	5.8774ms	2	2.9387ms	2.5962ms	3.2812ms	[CUDA memcpy DtoH]
API:	90.99%	1.69655s	2	848.27ms	699.40us	1.69585s	cudaMalloc
	7.10%	132.39ms	200	661.97us	423.90us	7.6760ms	cudaDeviceSynchronize
	0.96%	17.932ms	4	4.4831ms	3.1292ms	7.7131ms	cudaMemcpy
	0.45%	8.4117ms	2	4.2059ms	2.4000us	8.4093ms	cudaEventCreate
	0.43%	8.0093ms	200	40.046us	9.9000us	1.0583ms	cudaLaunchKernel
	0.04%	796.50us	2	398.25us	302.30us	494.20us	cudaFree
	0.01%	213.70us	200	1.0680us	200ns	31.300us	cudaGetLastError
	0.00%	90.700us	1	90.700us	90.700us	90.700us	cudaEventSynchronize
	0.00%	81.700us	101	808ns	200ns	31.300us	cuDeviceGetAttribute
	0.00%	30.100us	2	15.050us	10.700us	19.400us	cudaEventRecord
	0.00%	5.7000us	2	2.8500us	1.3000us	4.4000us	cudaEventDestroy
	0.00%	4.3000us	1	4.3000us	4.3000us	4.3000us	cuDeviceGetPCIBusId
	0.00%	3.9000us	1	3.9000us	3.9000us	3.9000us	cudaEventElapsedTime
	0.00%	2.3000us	3	766ns	400ns	1.4000us	cuDeviceGetCount
	0.00%	1.6000us	2	800ns	300ns	1.3000us	cuDeviceGet
	0.00%	1.5000us	1	1.5000us	1.5000us	1.5000us	cuDeviceGetName
	0.00%	800ns	1	800ns	800ns	800ns	cuDeviceTotalMem
	0.00%	400ns	1	400ns	400ns	400ns	cuDeviceGetUuid

T4:

Execute the CPU-only reference version...

- Elapsed time: 36.761 s

Execute GPU kernel...

- Elapsed time: 1741.090 ms

Max Error = 0.000015

The Max Error of 0.00002 is within ACCEPTABLE bounds.

- Total elapsed time: 46.074 s

==5152== Profiling application: /content/drive/MyDrive/Colab Notebooks/HPC-CUDA/build/cuda

==5152== Profiling result:

Type	Time(%)	Time	Calls	Avg	Min	Max	Name
GPU:	67.30%	1.73103s	200	8.6551ms	8.0181ms	11.542ms	step_kernel_mod(int, int, float, float*, float*)
	16.79%	431.89ms	2	215.95ms	203.45ms	228.45ms	[CUDA memcpy HtoD]
	15.91%	409.13ms	2	204.56ms	201.12ms	208.01ms	[CUDA memcpy DtoH]
API:	62.34%	1.73679s	200	8.6840ms	8.0240ms	11.548ms	cudaDeviceSynchronize
	30.23%	842.24ms	4	210.56ms	201.46ms	228.69ms	cudaMemcpy
	7.16%	199.38ms	2	99.688ms	144.08us	199.23ms	cudaMalloc
	0.13%	3.7350ms	200	18.674us	5.4010us	429.51us	cudaLaunchKernel
	0.13%	3.6890ms	2	1.8445ms	870.86us	2.8181ms	cudaFree
	0.01%	168.47us	114	1.4770us	185ns	65.476us	cuDeviceGetAttribute
	0.00%	67.106us	200	335ns	105ns	674ns	cudaGetLastError
	0.00%	33.409us	2	16.704us	10.078us	23.331us	cudaEventRecord
	0.00%	27.898us	2	13.949us	1.1280us	26.770us	cudaEventCreate
	0.00%	12.296us	1	12.296us	12.296us	12.296us	cuDeviceGetName
	0.00%	6.8260us	1	6.8260us	6.8260us	6.8260us	cuDeviceGetPCIBusId
	0.00%	4.4120us	1	4.4120us	4.4120us	4.4120us	cudaEventSynchronize
	0.00%	3.6190us	1	3.6190us	3.6190us	3.6190us	cudaEventElapsedTime
	0.00%	3.3080us	2	1.6540us	1.1110us	2.1970us	cudaEventDestroy
	0.00%	1.6790us	3	559ns	233ns	1.2100us	cuDeviceGetCount
	0.00%	716ns	2	358ns	179ns	537ns	cuDeviceGet
	0.00%	529ns	1	529ns	529ns	529ns	cuDeviceTotalMem
	0.00%	372ns	1	372ns	372ns	372ns	cuModuleGetLoadingMode
	0.00%	302ns	1	302ns	302ns	302ns	cuDeviceGetUuid

3.6 Results

The GPU simulation results were compared with the CPU version, and the speedup was calculated as:

$$\text{speedup} = \frac{T_{\text{CPU}}}{T_{\text{GPU}}}$$

GPU	T_{CPU} (ms)	T_{GPU} (ms)	Speedup
950M	34967	15086	x2.32
T4	37178	1645	x22.60

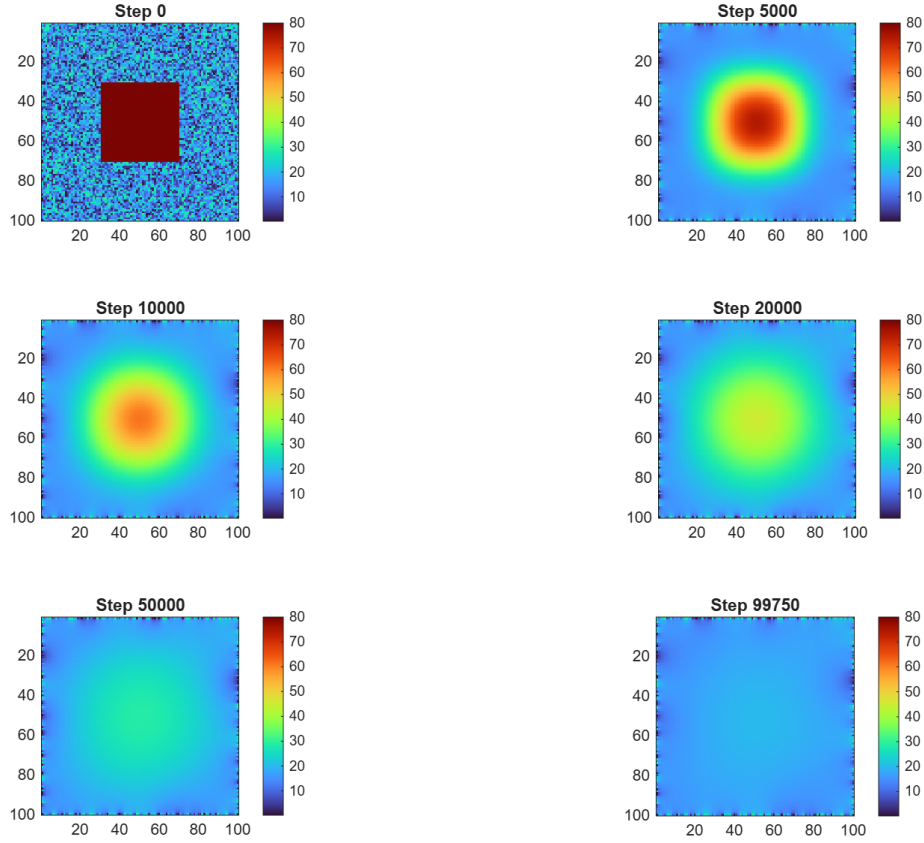
4 Conclusions

4.1 Realistic System Simulation

Following the analyses described in the previous section, I decided to extend the project with a real graphical simulation, in order to demonstrate that the numerical simulation is correct.

The parameters used for the real simulation are:

- $nstep = 100'000$
- $step = 250$
- $ni = 100$
- $nj = 100$
- $tfac = 0.01f$
- $size = ni \times nj = 10'000$



4.2 Considerations

GPU parallelization enabled a significant speedup, demonstrating the effectiveness of this approach compared to the CPU implementation.

The maximum error between GPU and CPU results was computed to verify correctness. In the current simulation, the maximum observed error remained within acceptable limits (0.0005).

These results confirm that the CUDA parallelization substantially reduces execution time relative to the sequential version while preserving numerical accuracy